Deep Learning March 13, 2026

1 Assignment description

- Submission deadline: *March 23, 2026, 12:00.*
- Grades released: *March 30, 2026, 12:00.*
- The assignment consists of two parts: main questions and resit questions. For the first submission, only the main questions must be completed.

  Failure to submit solutions to the main questions by the deadline results in loss of eligibility for the resit.
- If the main part is failed, one resit opportunity is available by submitting solutions to the resit questions. The resit deadline is *April 8, 2026, 11:00.*
- *Strict* format requirements:
  (a) Minimum font size: 11pt.
  (b) There is no strict page limit; however, explanations should be clear and concise.
  (c) Submit via Canvas Assignments and include:
    - a written report only in PDF format;
    - a zip file containing the code required to replicate the results.

2 Main questions

This section contains three main questions for the first sit. The first section covers the implementation of a deep learning pipeline from scratch. The focus is on the engineering of these models, so the most important mathematical details are provided. In the second section you will consider several issues relating to the gradient calculations to help you intuitively understand why certain developments occurred for deep learning models.

Notation: All notation, unless otherwise stated, follows the lecture.

Question 1 (60 points)

In this section, you will implement a small neural network from scratch and train it to solve a simple binary classification task. You are not allowed to use deep learning packages such as `PyTorch` for this assignment, as the learning objective is for you to be able to implement simple models yourself. You may use other packages though, and we recommend implementing it using `NumPy`. First, you will implement the layer and forward pass functions of a simple Multi-layer Perceptron (MLP). Then, you will implement the backward pass using the derivative functions of these layers. You will use mini-batch gradient descent to train this network, meaning that layer functions should operate over stacked tensors rather than single vectors.

1a. Dataset

Generate a two-dimensional binary classification dataset using the `make_moons` function from `scikit-learn`:

```
from sklearn.datasets import make_moons
X, y = make_moons(n_samples=300, noise=0.2, random_state=80)
```

This produces 300 samples with two features (x_1, x_2) and binary labels $y \in \{0, 1\}$. The two classes form interleaved half-moon shapes that are not linearly separable. Produce a scatter plot of the dataset, colouring each point by its class label. Then, split the data into an 80/20 train–validation split (we do not perform testing on this simple problem to avoid unnecessary implementation overhead). Ensure that splits are reproducible using fixed random states.

1b. Forward Pass

Bear in mind that layers should ingest mini-batches, and as such layers should be able to handle batch size ≥ 1. You will implement a two-layer MLP with the following fixed architecture:

Layer	**Units**	**Activation**
Input	2	—
Hidden layer	16	Sigmoid
Output layer	1	Sigmoid

Implement each layer. You are free to implement this model in a way that suits you. We suggest using classes to help track properties such as gradients, along the lines of the following interface:

```
class LinearLayer:
    def __init__(self, n_in, n_out): ...
    def forward(self, x): ...          # returns pre-activation z = xW + b


class SigmoidLayer:
    def forward(self, z): ...          # returns sigmoid(z)
```

Randomly initializing weights will often result in poor performance and troubles with initial weight updates (e.g. due to tiny gradients from very small initializations). Instead, you should set weights such that they allow for initial gradient updates to be effective. To do so, use the following weight initialisation scheme (Xavier uniform):

$$W \sim \mathcal{U}\Big(-\sqrt{\tfrac{6}{n_{\text{in}}+n_{\text{out}}}},\ \sqrt{\tfrac{6}{n_{\text{in}}+n_{\text{out}}}}\Big), \qquad b = \mathbf{0}.$$

Use binary cross-entropy as the loss function:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\big[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)\big].$$

Verify your forward pass by running a single forward pass on the full training set and confirming that the output $\hat{y}$ has the correct shape and lies in $(0, 1)$.

1c. Backward Pass

Reminder: Backpropagation. Recall from the lecture that backpropagation applies the chain rule layer by layer in reverse order. For a loss $\mathcal{L}$ and a layer with pre-activation $z = xW + b$ and the sigmoid-activated output $a = f(z)$, the key quantities in mini-batch form are:

$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial a} \odot f'(z), \qquad \frac{\partial \mathcal{L}}{\partial W} = x^\top \frac{\partial \mathcal{L}}{\partial z}, \qquad \frac{\partial \mathcal{L}}{\partial b} = \mathbf{1}^\top \frac{\partial \mathcal{L}}{\partial z}, \qquad \frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial z} W^\top.$$

The last expression passes the gradient upstream to the previous layer. For the combined sigmoid output and binary cross-entropy loss, the upstream gradient simplifies conveniently to $\partial\mathcal{L}/\partial z_{\text{out}} = \hat{y} - y$.

Extend each of your layer classes with a `backward` method. The table below summarises the full gradient flow through the network, from the loss back to the first layer. Each row corresponds to one step in the backward pass, and should map directly onto one method call in your implementation.

Step	Layer	Receives	Computes & returns
1	Output (`sigmoid` + BCE)	$\hat{y}$, y	$\partial\mathcal{L}/\partial z_2 = \hat{y} - y$
2	`LinearLayer` 2	$\partial\mathcal{L}/\partial z_2$	updates W_2, b_2; returns $\partial\mathcal{L}/\partial a_1$
3	`SigmoidLayer`	$\partial\mathcal{L}/\partial a_1$	$\partial\mathcal{L}/\partial z_1 = \partial\mathcal{L}/\partial a_1 \odot \sigma'(z_1)$
4	`LinearLayer` 1	$\partial\mathcal{L}/\partial z_1$	updates W_1, b_1; returns $\partial\mathcal{L}/\partial x$

What to cache and when. Each layer's `backward` method needs values that were computed during the forward pass. A good rule of thumb is: cache anything in `forward` that appears in the gradient expressions above. Concretely:

- `LinearLayer` needs x (the input it received) to compute $\partial\mathcal{L}/\partial W = x^\top \partial\mathcal{L}/\partial z$.
- `SigmoidLayer` needs z (the pre-activation) to evaluate $\sigma'(z)$.

Store these as instance attributes in `forward` (e.g. `self.x = x`) so they are available when `backward` is called.

Applying the gradients. Once you have computed $\partial\mathcal{L}/\partial W$ and $\partial\mathcal{L}/\partial b$ inside `LinearLayer.backward`, update the weights with a single gradient descent step:

$$W \leftarrow W - \eta \cdot \frac{\partial\mathcal{L}}{\partial W}, \qquad b \leftarrow b - \eta \cdot \frac{\partial\mathcal{L}}{\partial b},$$

where η is the learning rate. Do this *before* returning the upstream gradient $\partial\mathcal{L}/\partial x$, so that the weights are updated on every call to `backward`. Note that `SigmoidLayer` has no parameters to update, it only transforms and passes the gradient.

Gradient check. Before proceeding, verify your implementation using numerical differentiation. For a small subset of weights θ_j, confirm that:

$$\frac{\partial\mathcal{L}}{\partial\theta_j} \approx \frac{\mathcal{L}(\theta_j + \varepsilon) - \mathcal{L}(\theta_j - \varepsilon)}{2\varepsilon}, \qquad \varepsilon = 10^{-5}.$$

Report the maximum absolute difference across the checked weights. A value below 10^{-5} indicates your implementation is correct.

The expected tensor shapes are given below as an additional sanity check.

Gradient	**Shape**	**Description**
$\partial\mathcal{L}/\partial W_2$	$(16, 1)$	Output layer weights
$\partial\mathcal{L}/\partial b_2$	$(1,)$	Output layer bias
$\partial\mathcal{L}/\partial W_1$	$(2, 16)$	Hidden layer weights
$\partial\mathcal{L}/\partial b_1$	$(16,)$	Hidden layer bias

1d. Training

Train the network using mini-batch gradient descent with the following settings:

Hyperparameter	**Value**
Learning rate	0.05
Batch size	32
Epochs	200

At each epoch, shuffle the training data, then re-form the mini-batches. You may change parameters as you see fit. After training, produce the following:

(i) A plot of training loss and validation loss against epoch number on the same axes.

(ii) A visualisation of the learned decision boundary overlaid on the validation set scatter plot. To do this, evaluate the network on a fine grid covering $[-2, 3] \times [-1.5, 2]$ and use a filled contour plot at threshold 0.5.

(iii) The final validation accuracy (percentage of correctly classified validation points).

Lastly, comment briefly on the shape of the loss curves. Does the network successfully learn the non-linear boundary? If training loss decreases but validation loss plateaus or rises, what might this indicate?

Question 2 (40 points)

In this section, you will explore some of the theoretical properties of deep neural networks, with a focus on how gradient flow behaves during backpropagation and how architectural choices can mitigate common failure modes.

Vanishing Gradients

When training deep networks with backpropagation, the gradient of the loss with respect to early-layer weights is a product of many terms — one per layer. If each of those terms is smaller than 1 in magnitude, the product shrinks exponentially with depth, making early layers learn extremely slowly or not at all. This is known as the *vanishing gradient problem.*

Consider a simplified chain of n sigmoid activations applied sequentially to a scalar input x:

$$a_k = \sigma(a_{k-1}), \qquad k = 1, \ldots, n, \qquad a_0 = x,$$

By the chain rule, the gradient of the final output a_n with respect to the input x is:

$$\frac{\partial a_n}{\partial x} = \prod_{k=1}^{n} \sigma'(a_{k-1}).$$

The sigmoid derivative satisfies $\sigma'(z) \leq 1/4$ for all $z \in \mathbb{R}$. Using this bound, show that:

$$\left| \frac{\partial a_n}{\partial x} \right| \leq \left(\frac{1}{4} \right)^n.$$

Question 2a What does this imply about the gradient as n grows? Evaluate the upper bound $(1/4)^n$ numerically for $n \in \{5, 10, 20\}$. Summarise your findings in a small table and comment on what this means for training a network of depth 20 with sigmoid activations.

2b. Sigmoid vs. ReLU

The Rectified Linear Unit (ReLU) is defined as $\mathrm{ReLU}(z) = \max(0, z)$, with derivative:

$$\mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0, \\ 0 & z \leq 0. \end{cases}$$

Question 2b On a single figure with two subplots, plot the derivative $\sigma'(z)$ of the sigmoid and the derivative $\mathrm{ReLU}'(z)$ of the ReLU over the range $z \in [-4, 4]$. Label both axes clearly. With reference to your plots, explain briefly why ReLU activations are substantially less susceptible to the vanishing gradient problem than sigmoid activations.

The Dying ReLU Problem

While ReLU activations avoid vanishing gradients for active neurons, it introduces a different failure mode: a neuron whose pre-activation z is negative for every input in the dataset will always output zero, receive a zero gradient, and consequently never update its weights. Such a neuron is said to be *dead*.

Question 2c. Consider a single ReLU neuron with scalar weight w, bias b, and input x. Write down the gradient $\partial\mathcal{L}/\partial w$ when $wx+b<0$. What does this imply for weight updates? What are the consequences during the training of the network, especially during later stages of training?

Residual Connections

A residual block, as introduced by He et al. (2016) for Convolutional Neural Networks (CNNs), transforms its input x as follows:

$$\text{out} = f(x + g(x)),$$

where $g(\cdot)$ represents one or more intermediate layers (e.g. linear + activation) and $f(\cdot)$ is the activation applied after the skip connection is added. The term x on its own is the *skip connection*: it bypasses g and is added directly to its output. Notice that the derivative form can be written as follows:

$$\frac{\partial\mathcal{L}}{\partial x} = \frac{\partial\mathcal{L}}{\partial\,\text{out}} \cdot f'(x + g(x)) \cdot \big(1 + g'(x)\big).$$

Question 2d. Identify the term in the expression above that is absent in a plain (non-residual) layer. Explain briefly why this term guarantees that the gradient $\partial\mathcal{L}/\partial x$ cannot vanish through this block due to $g'(x)$ alone, even if $g'(x) \approx 0$.

Residual blocks emerging in the context of computer vision is not a coincidence. As networks became deeper to handle increasingly complex visual tasks, moving from a handful of layers to potentially hundreds, a surprising problem appeared: deeper networks were sometimes performing *worse* than shallower ones, even on the training set. This is not overfitting, because a deeper network should in principle be able to replicate the behavior of a shallower one simply by learning identity mappings in its extra layers. Yet in practice, deeper models failed to do so.

In a plain deep network with no skip connections, the gradient reaching an early layer has passed multiplicatively through every subsequent layer, each one contributing a factor of $g'(x)$ with no additive offset. Early and late layers are therefore tightly coupled: neither can make reliable progress without the other already having learned something useful.

Question 2e. With reference to the derivative form shown above, explain intuitively why skip connections help to decouple the gradient signal reaching early layers from the current state of the layers above them. Why did this in particular aid deep CNNs? (Hint: the skip connection introduces an additive 1 into the gradient path. Reason about what this allows gradient updates to do versus the plain MLP blocks without skip connections).

Best of luck, and have fun!

3 Resit questions

Question R1

Generate a two-dimensional binary classification dataset using the `make_circles` function from `scikit-learn`:

Then, repeat the steps of section 1 with the following architecture:

Layer	**Units**	**Activation**
Input	2	—
Hidden layer	32	ReLU
Output layer	1	Sigmoid

Produce the same outputs as in Section 1d: a loss curve, a decision boundary plot over the range $[-1.5, 1.5] \times [-1.5, 1.5]$, and the final validation accuracy. Answer the same question posed in 1d: do the loss curves behave as expected, and does the network successfully recover the non-linear boundary?

Question R2

Just as gradients can vanish when the terms in their product are smaller than 1 in magnitude, they can also *explode* when those terms are larger than 1. In this case the product grows exponentially with depth, causing weight updates to become unstably large and training to diverge. This is known as the *exploding gradient problem*, and it is the mirror image of the vanishing gradient problem explored in Section 2a.

Consider a simplified chain of n linear neurons with no activation function, each with a scalar weight $w > 1$. The output of layer k is $a_k = w \cdot a_{k-1}$, with $a_0 = x$ as the scalar input. By the chain rule, the gradient of the final output a_n with respect to the input x is:

$$\frac{\partial a_n}{\partial x} = \prod_{k=1}^{n} w = w^n.$$

Using the expression above, show that for $w > 1$:

$$\left| \frac{\partial a_n}{\partial x} \right| = w^n \to \infty \quad \text{as } n \to \infty.$$

What does this imply about gradient updates in deep networks with weights initialised to values greater than 1?

Question R3

You are provided with a Jupyter notebook containing two pre-built model classes — `PlainMLP` and `ResNetMLP` — as well as a pre-generated dataset. The notebook is available at the following link: (made available once resit period starts)

`PlainMLP` is a fully-connected network with 6 hidden layers of 32 units each and ReLU activations, with no skip connections. `ResNetMLP` has the same depth and width, but every two hidden layers are wrapped in a residual block of the form discussed in Section 2c:

$$\text{out} = \text{ReLU}(x + g(x)),$$

where $g(\cdot)$ consists of two linear layers with a ReLU activation in between. Both models are initialised with the same random seed. The dataset is a six-class spiral pattern in two dimensions, which requires deep representations to fit well and is therefore designed to expose differences in training dynamics between the two architectures.

Discuss the loss curves of both models. What do you observe? Can you reason why residual blocks make such a difference during training?

References

He, K., X. Zhang, S. Ren, and J. Sun (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778.